

გაკვეთილი 20 mod 3 (3.3)

გაკვეთილი 8 — მონაცემთა ეფექტური დამუშავება და ტრანსფორმაცია მასივების გამოყენებით

ეს გაკვეთილი კარგი გაგრძელებაა მე-5 და მე-6 გაკვეთილების. აქ ახალი მეთოდების დიდი რაოდენობით დამატება არ გვჭირდება — მოსწავლეები უკვე ნასწავლ `filter()`, `map()`, `find()`, `sort()` -ს ერთად გამოიყენებენ რეალური ამოცანების გადასაჭრელად.

გაკვეთილის მიზანი

გაკვეთილის ბოლოს მოსწავლემ უნდა შეძლოს:

- მასივიდან საჭირო მონაცემების შერჩევა;
- მონაცემების გარდაქმნა;
- კონკრეტული ელემენტის მოძებნა;
- მონაცემების დალაგება;
- რამდენიმე მოქმედების თანმიმდევრულად შესრულება;
- ფუნქციის გამოყენება მასივის დასამუშავებლად;
- საწყისი და დამუშავებული მონაცემების ერთმანეთისგან გარჩევა.

1. რას ნიშნავს მონაცემთა დამუშავება?

ვთქვათ გვაქვს რიცხვების მასივი:

```
let numbers = [15, 80, 32, 120, 5, 64];
```

ეს არის ჩვენი საწყისი მონაცემები.

შეიძლება დაგვჭირდეს:

- მხოლოდ 50-ზე მეტი რიცხვები;
- ყველა რიცხვის გაორმაგება;
- ზრდადობით დალაგება;
- პირველი 100-ზე მეტი რიცხვის პოვნა.

ამ ყველაფერს ეწოდება მონაცემთა დამუშავება.

2. რას ნიშნავს მონაცემთა ტრანსფორმაცია?

ტრანსფორმაცია ნიშნავს მონაცემის შეცვლას ან მისგან ახალი მონაცემის შექმნას.

მაგალითად გვაქვს:

```
let numbers = [10, 20, 30];
```

გვინდა ყველა რიცხვი გავაორმაგოთ:

```
let doubled = numbers.map(function(num){  
    return num * 2;  
});  
  
console.log(doubled);
```

შედეგი:

```
[20, 40, 60]
```

თავდაპირველი მასივი იყო:

```
[10, 20, 30]
```

ხოლო ტრანსფორმაციის შემდეგ მივიღეთ:

```
[20, 40, 60]
```

3. გავიხსენოთ `filter()`

`filter()` გამოიყენება მაშინ, როდესაც მასივიდან მხოლოდ გარკვეული ელემენტების დატოვება გვინდა.

მაგალითად:

```
let numbers = [10, 55, 20, 80, 35, 100];

let bigNumbers = numbers.filter(function(num){
  return num > 50;
});

console.log(bigNumbers);
```

შედეგი:

```
[55, 80, 100]
```

მარტივი დასამახსოვრებელი წესი:

`filter()` — რომელი ელემენტები დავტოვო?

4. გავიხსენოთ `map()`

`map()` გამოიყენება მაშინ, როდესაც ყველა ელემენტის შეცვლა/გარდაქმნა გვინდა.

```
let numbers = [5, 10, 15];

let newNumbers = numbers.map(function(num){
  return num + 10;
});

console.log(newNumbers);
```

შედეგი:

```
[15, 20, 25]
```

დასამახსოვრებლად:

`map()` — როგორ შევცვალო თითოეული ელემენტი?

5. გავიხსენოთ `find()`

`find()` ეძებს პირველ ელემენტს, რომელიც მოცემულ პირობას აკმაყოფილებს.

```
let numbers = [20, 40, 120, 150, 60];

let result = numbers.find(function(num){
  return num > 100;
});

console.log(result);
```

შედეგი:

```
120
```

150-იც 100-ზე მეტია, მაგრამ `find()` უკვე 120-ის პოვნისას ჩერდება.

დასამახსოვრებლად:

`find()` — იპოვე პირველი შესაბამისი ელემენტი.

6. გავიხსენოთ `sort()`

რიცხვების ზრდადობით დასალაგებლად:

```
let numbers = [50, 10, 80, 20];

numbers.sort(function(a, b){
  return a - b;
});

console.log(numbers);
```

შედეგი:

```
[10, 20, 50, 80]
```

კლებადობით:

```
numbers.sort(function(a, b){
  return b - a;
```

```
});
```

შედეგი:

```
[80, 50, 20, 10]
```

7. მთავარი ნაწილი — რამდენიმე მოქმედების ერთად გამოყენება

ახლა უკვე ცალ-ცალკე აღარ ვაკეთებთ ყველაფერს.

გვაქვს:

```
let numbers = [20, -5, 70, 10, -15, 100];
```

ამოცანა:

დატოვე მხოლოდ დადებითი რიცხვები და შემდეგ გააორმაგე ისინი.

პირველად ვფილტრავთ:

```
let positive = numbers.filter(function(num){  
  return num > 0;  
});
```

მივიღეთ:

```
[20, 70, 10, 100]
```

ახლა ვცვლით:

```
let doubled = positive.map(function(num){  
  return num * 2;  
});
```

მივიღეთ:

```
[40, 140, 20, 200]
```

სრული კოდი:

```
let numbers = [20, -5, 70, 10, -15, 100];

let positive = numbers.filter(function(num){
  return num > 0;
});

let doubled = positive.map(function(num){
  return num * 2;
});

console.log(positive);
console.log(doubled);
```

ეს უკვე არის მონაცემთა დამუშავება და ტრანსფორმაცია.

8. filter() + map() + sort()

გვაქვს:

```
let numbers = [30, -5, 80, 10, 60, -20];
```

გვინდა:

1. მხოლოდ დადებითი რიცხვები
2. თითოეული გაგაორმაგოთ
3. შედეგი ზრდადობით დავალაგოთ

```
let positive = numbers.filter(function(num){
  return num > 0;
});

let doubled = positive.map(function(num){
  return num * 2;
});

doubled.sort(function(a, b){
  return a - b;
});
```

```
console.log(doubled);
```

შედეგი:

```
[20, 60, 120, 160]
```

9. ფუნქციის გამოყენება მონაცემების დასამუშავებლად

შეგვიძლია ეს ყველაფერი ფუნქციაში მოვათავსოთ.

```
function processNumbers(numbers){  
  
    let result = numbers.filter(function(num){  
        return num > 0;  
    });  
  
    result = result.map(function(num){  
        return num * 2;  
    });  
  
    result.sort(function(a, b){  
        return a - b;  
    });  
  
    return result;  
}  
  
let numbers = [30, -5, 80, 10, 60, -20];  
  
let answer = processNumbers(numbers);  
  
console.log(answer);
```

აქ:

```
processNumbers(numbers)
```

იღებს მასივს.

შემდეგ:

```
filter()
```

ტოვებს დადებითებს.

შემდეგ:

```
map()
```

აორმაგებს.

შემდეგ:

```
sort()
```

ალაგებს.

ბოლოს:

```
return result;
```

აბრუნებს საბოლოო მასივს.

10. მომხმარებლისგან მონაცემების მიღება

ახლა უფრო პრაქტიკული მაგალითი გავაკეთოთ.

მომხმარებელმა შეიყვანოს 6 რიცხვი:

```
let numbers = [];  
  
for(let i = 0; i < 6; i++){  
    let num = Number(prompt("შეიყვანე რიცხვი"));  
    numbers.push(num);  
}  
  
console.log(numbers);
```

შემდეგ დავამუშავოთ:

```
let positive = numbers.filter(function(num){
    return num > 0;
});

let doubled = positive.map(function(num){
    return num * 2;
});

doubled.sort(function(a, b){
    return a - b;
});

console.log("საწყისი მასივი:", numbers);
console.log("დადებითი:", positive);
console.log("გაორმაგებული და დალაგებული:", doubled);
```

11. რეალური მაგალითი — ფასები

გვაქვს პროდუქციის ფასები:

```
let prices = [20, 150, 70, 200, 45, 110];
```

გვინდა:

50 ლარზე მეტი პროდუქტების ფასები გავზარდოთ 10 ლარით.

ჯერ:

```
let expensive = prices.filter(function(price){
    return price > 50;
});
```

მივიღებთ:

```
[150, 70, 200, 110]
```

შემდეგ:

```
let newPrices = expensive.map(function(price){
    return price + 10;
});
```

```
});
```

მივიღებთ:

```
[160, 80, 210, 120]
```

სრული კოდი:

```
let prices = [20, 150, 70, 200, 45, 110];

let expensive = prices.filter(function(price){
  return price > 50;
});

let newPrices = expensive.map(function(price){
  return price + 10;
});

console.log(newPrices);
```

12. რეალური მაგალითი — მოსწავლეების ქულები

```
let scores = [45, 90, 67, 30, 100, 75];
```

გვინდა მხოლოდ ის ქულები, რომლებიც 51-ზე მეტია:

```
let passed = scores.filter(function(score){
  return score > 51;
});

console.log(passed);
```

შემდეგ დავალაგოთ კლებადობით:

```
passed.sort(function(a, b){
  return b - a;
});

console.log(passed);
```

შედეგი:

```
[100, 90, 75, 67]
```

13. find() -ის დამატება

იგივე ქულებიდან ვიპოვოთ პირველი ქულა, რომელიც 90-ზე მეტია:

```
let scores = [45, 90, 67, 30, 100, 75];

let highScore = scores.find(function(score){
  return score > 90;
});

console.log(highScore);
```

შედეგი:

```
100
```



External JavaScript

როგორც წინა გაკვეთილებში, კოდი ცალკე `script.js` ფაილში გვქვია.

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Array Processing</title>
</head>

<body>

  <h1>Array Processing</h1>

  <script src="script.js"></script>
```

```
</body>
</html>
```

script.js

```
let numbers = [];

for(let i = 0; i < 6; i++){

    let num = Number(prompt("შეიყვანე რიცხვი"));

    numbers.push(num);
}

let positive = numbers.filter(function(num){
    return num > 0;
});

let doubled = positive.map(function(num){
    return num * 2;
});

doubled.sort(function(a, b){
    return a - b;
});

let firstBig = doubled.find(function(num){
    return num > 100;
});

console.log("საწყისი:", numbers);
console.log("დადებითი:", positive);
console.log("გაორმაგებული:", doubled);
console.log("პირველი 100-ზე მეტი:", firstBig);
```



საკლასო სავარჯიშოები

1. მომხმარებლისგან მიიღე 8 რიცხვი. `filter()` -ით დატოვე მხოლოდ ლუწები და `map()` -ით თითოეული გაამრავლე 3-ზე.
2. შექმენი ფასების მასივი. დატოვე მხოლოდ 100 ლარზე მეტი ფასები, თითოეულს გამოაკელი 10 ლარი და შედეგი დაალაგე ზრდადობით.

3. მომხმარებლისგან მიიღე 7 რიცხვი. დატოვე მხოლოდ დადებითი რიცხვები, გააორმაგე ისინი და იპოვე პირველი მიღებული რიცხვი, რომელიც 50-ზე მეტია.
4. შექმენი ქულების მასივი. დატოვე მხოლოდ 51-ზე მეტი ქულები და დააღაგე კლებადობით.
5. შექმენი ფუნქცია:

```
function transformArray(arr)
```

ფუნქციამ უნდა დატოვოს მხოლოდ დადებითი რიცხვები, თითოეული გაამრავლოს 5-ზე და დააბრუნოს მიღებული მასივი.

შედარებით რთული სავარჯიშო

მომხმარებლისგან მიიღე 10 რიცხვი.

პროგრამამ უნდა:

- `filter()` -ით დატოვოს მხოლოდ დადებითი რიცხვები;
- `map()` -ით თითოეული დადებითი რიცხვი გაამრავლოს 2-ზე;
- `sort()` -ით დააღაგოს მიღებული მასივი ზრდადობით;
- `find()` -ით იპოვოს პირველი მიღებული რიცხვი, რომელიც 100-ზე მეტია;
- შექმნას ფუნქცია, რომელიც დაითვლის მიღებული მასივის ყველა ელემენტის ჯამს;
- გამოიტანოს საწყისი და საბოლოო შედეგები.

ჯამის ფუნქცია შეიძლება იყოს:

```
function calculateSum(arr){  
  
    let sum = 0;  
  
    for(let i = 0; i < arr.length; i++){  
        sum += arr[i];  
    }  
  
    return sum;  
}
```

 მთავარი, რაც მოსწავლემა უნდა დაიმახსოვროს

`filter()` → ვარჩევთ საჭირო ელემენტებს.

```
return num > 0;
```

`map()` → ვცვლით ელემენტებს.

```
return num * 2;
```

`find()` → ვეძებთ პირველ შესაბამის ელემენტს.

```
return num > 100;
```

`sort()` → ვალაგებთ ელემენტებს.

```
return a - b;
```

`function` → რამდენიმე მოქმედებას ერთ ბლოკში ვაერთიანებთ და საჭიროებისას ვიძახებთ.

`return` → ფუნქციიდან შედეგს ვაბრუნებთ.

ყველაზე მარტივი სქემა ბავშვებისთვის:

საწყისი მონაცემები → შერჩევა → გარდაქმნა → დალაგება/ძებნა → შედეგი

მაგალითად:

```
[10, -5, 30, 60]
  ↓ filter
[10, 30, 60]
  ↓ map × 2
[20, 60, 120]
  ↓ sort
[20, 60, 120]
  ↓ find > 100
120
```

ეს არის ზუსტად მე-8 გაკვეთილის მთავარი იდეა — ბავშვმა უკვე აღარ უნდა იცოდეს მხოლოდ ცალ-ცალკე რას აკეთებს `filter()`, `map()`, `find()` და `sort()`, არამედ უნდა შეძლოს მათი ერთ ამოცანაში სწორად გაერთიანება.